
Advanced AI Interview System

COMPREHENSIVE TECHNICAL RESEARCH, ENGINEERING, & SYSTEM
SPECIFICATION REPORT 

Author: Simarpreet Singh  

Date: June 22, 2026 

Project Code Name: interview-proctor-ai 

Document Class: Technical Whitepaper & Architectural Specification 

Status: Production Verified & Sealed 

Target Environment: Render / Vercel Stack / Multi-DB Sync 

Document Preface & Structure Map ■■

This project documentation serves as an exhaustive guide to the Advanced AI Interview System 🚀. The report has been systematically structured to outline the architectural foundations, requirement specifications 📄, core algorithm implementations 🧠, database models 🗄️, proctoring security systems 🛡️, and deployment configurations of the platform.

Chapter / Section 📁	Target Page 📄
Chapter 1: Project Overview & Objectives 🚀	Pages 3 - 6
Chapter 2: Literature Review & Technology Survey 📄	Pages 7 - 11
Chapter 3: System Requirements Analysis 📊	Pages 12 - 19
Chapter 4: System Architecture & Structural Design 🌐	Pages 20 - 24
Chapter 5: Backend Implementation Details ⚙️	Pages 25 - 36
Chapter 6: Front-End UI Engine & Real-Time Analytics 💻	Pages 37 - 41
Chapter 7: Recruiter Management & Dashboards 👥	Pages 42 - 46
Chapter 8: DevOps, Containerization & Operations ■■	Pages 47 - 50
Chapter 9: Testing, Verification & Quality Assurance ✅	Pages 51 - 54
Chapter 10: Future Scope & Conclusion 🎯	Pages 55 - 57

Chapter 10: Future Scope & Conclusion

10.1 Future Technical Enhancements

Future development plans for the platform include :

- **Local Speech Models** : Transitioning to local, optimized Whisper models to reduce reliance on external cloud APIs.
- **Real-Time Speech Analysis** : Implementing pitch and tone analysis to evaluate candidate confidence.
- **Dynamic Follow-Up Questions** : Configuring the AI to generate follow-up questions in real-time based on candidate answers.

10.2 Concluding Thoughts & Project Summary 🎯

The **Advanced AI Interview System** provides a scalable, automated solution for technical recruitment 🎯.

By integrating web technologies with machine learning APIs and client-side proctoring, the platform automates resume vetting 📄, interview simulation 🖋️, response evaluation 🗣️, and recruiter dashboard tracking 📊.

The system is designed with fallback features to ensure reliability in low-resource environments, making it a robust option for automated technical screenings.

10.3 References, Documentation & Bibliography

The following documentation and libraries were referenced during development :

- FastAPI Web Framework Documentation (<https://fastapi.tiangolo.com>)
 - Face-API.js TensorFlow Model Weights Repository (<https://github.com/justadudewhohacks/face-api.js>)
 - ReportLab PDF Compilation Library Documentation (<https://www.reportlab.com>)
 - OpenAI Whisper Speech-to-Text Model Details (<https://github.com/openai/whisper>)
 - SQLAlchemy Database Toolkit and ORM Mapper (<https://www.sqlalchemy.org>)
-

Chapter 1: Project Overview & Objectives

1.1 Project Background & Industry Context

The recruitment industry is undergoing a massive transformation powered by artificial intelligence and automated pipelines . Historically, the vetting of candidates for technical positions required heavy manual intervention. HR recruiters had to read hundreds of resumes manually , identify candidate skills, schedule initial phone screens, conduct mock technical interviews , and rate candidates based on subjective evaluations. This manual structure introduces significant bottlenecks, increases the cost-to-hire ratio, and slows down the talent acquisition lifecycle.

The **Advanced AI Interview System** was engineered to resolve these issues by providing an end-to-end automated mock interview room . By combining front-end facial analytics, web-based speech recording, and remote server-side evaluation, the system removes the human bottleneck. Candidates can complete preliminary technical screenings at any time from their home browser, while recruiters receive detailed portfolios containing performance grading , transcription dossiers, sentiment trends , and integrity reports . This enables organizations to screen thousands of applicants in parallel, significantly reducing cost and subjective bias.

1.2 Core System Goals and Objectives 🎯

The system was designed with four principal objectives in mind 🎯:

1. **Automated Vetting Precision** 📄: Implement highly accurate heuristics to parse resume PDFs and extract relevant metadata like names, qualifications, email addresses, phone numbers, age, and core skills. This data is mapped to specific domains to generate targeted questions.
2. **Real-Time Proctoring & Compliance** 🛡️: Prevent cheating during remote evaluations. The system must verify candidate presence, identify eye focus drift, detect tab switches, and log all violations with timestamps into the candidate database dossier.
3. **Semantic Evaluation Integrity** 🗣️: Employ state-of-the-art language models (such as Gemini and GPT) to grade speech transcripts. If internet connectivity or model APIs fail, the system must fallback to local TF-IDF models to maintain continuous operation.
4. **Actionable Admin Dashboards** 📊: Offer recruiters a web panel that visualizes score distributions, lists candidate statuses, hosts recorded video replies, and provides quick links for messaging candidates.

1.3 Scope, Boundaries, and Limitations 🌐

The operational scope of the platform is designed around web-based screening rooms 🌐. The system is optimized for mainstream modern browsers like Google Chrome, Microsoft Edge, and Mozilla Firefox. It is designed to handle primary technical fields including Computer Science 🖥️, AI/ML, Electrical Engineering, Mechanical Engineering, and Electronics.

However, there are critical architectural boundaries and limitations 🌐 :

- **Webcam & Microphones 📹:** Candidates must grant camera and microphone access. Lack of visual/audio feeds prevents the integrity and transcription features from functioning.
- **Local Processing Resource Limits ⚙️ :** Preloading large local AI models on low-tier hosting environments (e.g. Render Free Tier) can trigger Out of Memory (OOM) crashes. To mitigate this, the system disables heavy neural networks like FAISS and Vosk locally, substituting lightweight APIs and local fallback logic.
- **Client-Side Dependencies 🤖:** Client-side proctoring relies on Face-API.js. Heavy browser load or dark camera feeds can reduce face-detection accuracy, which can occasionally lead to false proctoring flags.

1.4 Market Need and Feasibility Analysis

A detailed feasibility study was conducted to validate the platform's viability across technical, operational, and financial dimensions :

- **Technical Feasibility**  : The stack uses FastAPI for high-throughput async connections, combined with SQLite for local dev and PostgreSQL for production. Front-end face modeling is handled in client browsers using WebGL-accelerated Tensorflow.js models (Face-API.js), minimizing CPU load on application servers.
- **Operational Feasibility**  : Recruiters do not need machine learning expertise to use the dashboard. The admin dashboard uses a visual UI backed by Highcharts. Candidates can launch the mock room in one click without installing external software.
- **Financial Feasibility**  : By leveraging free-tier APIs (e.g. Gemini 1.5 Flash, Cloudinary free tiers) and local open-source fallbacks (TF-IDF, energy VAD), hosting costs are kept low. The platform is highly scalable, supporting thousands of concurrent rooms at minimal cost.

Chapter 2: Literature Review & Technology Survey

2.1 Comparison of Speech-to-Text Transcription Paradigms

High-accuracy speech-to-text is essential for evaluating candidate responses . Three paradigms were evaluated during development:

- **Local Vosk Transcription**  : An open-source, offline speech recognition engine. Vosk works without an internet connection and has a small memory footprint. However, its accuracy is heavily dependent on acoustic models, making it susceptible to background noise and varying accents.
- **Local Whisper Model (Base)**  : OpenAI's Whisper model run locally via PyTorch. While highly accurate, loading the model requires substantial CPU and RAM, which makes it unsuitable for OOM-constrained hosting providers.
- **Cloud-Based Groq Whisper (whisper-large-v3-turbo)**  : A cloud-hosted API using custom LPU (Language Processing Unit) hardware. Groq transcribes audio files in milliseconds with high accuracy. The backend employs this API as its primary transcriber, sending WebM recordings via POST and receiving text outputs.

2.2 Evaluation of Face & Emotion Recognition Models 🤖

Integrity tracking requires continuous face monitoring 🤖. We evaluated several models for this task:

- **Server-Side DeepFace / OpenCV Processing** ⚙️ : Sending video frames to the server for processing. While accurate, this approach consumes significant network bandwidth and server CPU, making it unscalable for concurrent users.
- **Client-Side Face-API.js (TensorFlow.js Wrapper)** 🖥️ : A browser-based solution that performs face tracking and emotion analysis locally. It uses the GPU via WebGL to analyze frames in real-time, detecting emotions like neutral, happy, sad, angry, and thinking.

By performing face detection on the client and sending only periodic logs to the server, the platform minimizes backend compute costs and scales efficiently.

2.3 Database Architectures: SQLite vs PostgreSQL

The platform supports dual database setups to balance ease of development with production scalability :

- **SQLite (Local Staging)** : A zero-configuration database stored in a single file. SQLite is ideal for local development and offline testing, requiring no background services. However, it lacks support for high concurrency and row-level locking.
- **PostgreSQL (Production Environment)** : A robust relational database capable of handling heavy parallel transactions.

To support both, the system uses the ****SQLAlchemy ORM****. The application auto-detects the database from connection strings and handles database mirroring: when PostgreSQL is the primary database, the application automatically mirrors transactions to a local SQLite backup to prevent data loss.

2.4 Task Queues and Asynchronous Workers ⚙️

Processing audio transcriptions and video uploads synchronously on the main web thread can cause requests to time out ⚙️. We evaluated two asynchronous options:

- **FastAPI BackgroundTasks ⚡**: A simple built-in utility that runs tasks after returning a response. While lightweight, it runs in the same process, meaning CPU-heavy tasks can block the event loop.
- **Celery Workers with Redis Broker 📦**: A distributed task queue that offloads tasks to separate worker processes.

The system uses Celery for heavy tasks (transcription and Cloudinary uploads) in production. Redis acts as the message broker, queuing incoming tasks and delivering them to workers, keeping the main FastAPI thread highly responsive.

2.5 Web Framework Comparison: FastAPI, Flask, and Django ⚡

Choosing the right backend framework was critical for handling async media uploads and WebRTC signaling ⚡:

- **Django** 📁: A comprehensive framework with built-in admin tools. However, its synchronous architecture makes it less suited for high-concurrency async operations.
- **Flask** 📁: A lightweight, synchronous micro-framework. While simple, it requires third-party plugins to handle async operations.
- **FastAPI** 🚀: Built on ASGI (Uvicorn), FastAPI natively supports async operations. It automatically generates OpenAPI documentation, enforces type safety using Pydantic, and handles high-concurrency requests efficiently, making it the ideal choice for this system.

Chapter 3: System Requirements Analysis

3.1 Functional Requirements: Authentication and Profile Calibration

The system must support the following authentication and profile requirements :

- ****FR-1 (Registration)**** : Candidates must be able to register accounts using a unique username and password.
- ****FR-2 (Secure Login)**** : The system must authenticate users, verify credentials, and issue a JWT token with a 24-hour expiration.
- ****FR-3 (Access Revocation)**** : The API must validate JWT tokens on protected routes. If the admin revokes a user's access, the system must immediately block them.
- ****FR-4 (Profile Calibration)**** : Candidates must provide calibration details (age, target experience level) to allow the system to adapt question difficulty.

3.2 Functional Requirements: Resume Upload & Parsing Engine

The resume processing module must support the following requirements :

- ****FR-5 (Multi-Format Support)**** : The system must accept PDF and plain text resume uploads via a multi-part form.
 - ****FR-6 (Cloud and Local Storage)**** : Uploaded resumes must be saved locally on disk, with optional auto-upload to Cloudinary if configured.
 - ****FR-7 (Metadata Extraction)**** : The parser must extract candidate contact info, qualifications, and skills.
 - ****FR-8 (Education Mapping)**** : The parser must map qualifications to standardized categories (e.g. B.Tech CSE, B.Sc Data Science, BCA).
-

3.3 Functional Requirements: Interview Room Simulation

The simulation page must support the following interactive requirements :

- ****FR-9 (Webcam Display)**** : The UI must stream the candidate's webcam feed onto a visual canvas.
 - ****FR-10 (Speech Recording)**** : The client must record audio response chunks (WebM/WAV) using the MediaRecorder API.
 - ****FR-11 (Dynamic Question Display)**** : The system must display questions sequentially, accompanied by audio readouts powered by Text-to-Speech (gTTS).
 - ****FR-12 (Submission Timer)**** : The UI must enforce response limits (e.g., 25-second answering timers) and automatically submit the answer when time expires.
-

3.4 Functional Requirements: Integrity & Proctoring

The system must implement proctoring requirements to ensure interview integrity  :

- ****FR-13 (Tab-Blur Tracking)****  : The client must monitor window focus. If a user switches tabs, the system must log a violation.
- ****FR-14 (Face Presence Check)****  : The proctoring engine must verify that a face is present in the webcam feed.
- ****FR-15 (Gaze Tracking)****  : The engine must detect if the candidate looks away from the screen, logging gaze deviations.
- ****FR-16 (Auto-Locking)****  : If the system detects repeated violations (e.g., three tab switches), it must flag the candidate's record as 'Compromised' in the database.

3.5 Functional Requirements: Evaluation and AI Grading 🧠

The grading engine must support the following scoring requirements 🧠:

- ****FR-17 (Speech Transcription)**** 🗣️: The backend must transcribe uploaded audio response chunks using the Whisper API.
- ****FR-18 (LLM Evaluation)**** 🤖: The system must grade transcripts using Gemini or GPT models, returning a score (0-100) and structured feedback.
- ****FR-19 (Rule-Based Grading Fallback)**** ⚙️: If LLM APIs are unreachable, the system must grade answers locally using TF-IDF similarity metrics.
- ****FR-20 (Sentiment Integration)**** 😊: Evaluation prompts should include the candidate's emotional state to adjust the final score.

3.6 Functional Requirements: Admin Dashboard Controls

The administrative dashboard must support the following management requirements :

- ****FR-21 (Dossier Visualization)**** : The dashboard must list registered users, average scores, primary emotions, and integrity statuses.
- ****FR-22 (Outreach Integration)**** : Admins must be able to launch Gmail or WhatsApp messages pre-filled with candidate evaluation details directly from the UI.
- ****FR-23 (Action Flags)****  : Admins must be able to approve, reject, or delete candidate profiles and toggle simulation access.
- ****FR-24 (PDF Export)****  : Admins must be able to export candidate performance records into a clean PDF summary report.

3.7 Non-Functional Requirements: Security & Compliance

The system must enforce the following security and operational compliance standards  :

- ****NFR-1 (Data Isolation)****  : Database credentials and API keys must be kept secure using environment variables (`.env`).
- ****NFR-2 (JWT Token Enforcement)****  : Protected endpoints must validate JWT tokens, rejecting requests with expired or missing signatures.
- ****NFR-3 (Input Sanitization)****  : Usernames must be sanitized before being used in file system paths to prevent path traversal attacks.
- ****NFR-4 (Secure Storage)****  : Resumes and video recordings must be stored securely, using Cloudinary HTTPS connections in production.

3.8 Non-Functional Requirements: Scalability & Performance

The system must meet the following performance and scalability targets :

- ****NFR-5 (Response Latency)****  : Page loads and API endpoints should load within 200ms under standard network conditions.
- ****NFR-6 (Async Transcription)****  : Transcription tasks must run asynchronously using Celery worker pools to prevent blocking the main server process.
- ****NFR-7 (Database Sync)****  : Data mirroring between primary and secondary databases must be transaction-safe.
- ****NFR-8 (Low Resource Footprint)****  : Local fallback operations must consume minimal memory to avoid OOM crashes on shared hosting environments.

Chapter 4: System Architecture & Structural Design

4.1 Multi-Tier System Topology & Core Components

The platform is structured as a multi-tier web application to separate user interfaces from core business logic and storage :

- **Presentation Layer (Frontend)** : Static HTML5, Vanilla CSS, and JavaScript. It uses libraries like `Face-API.js` for proctoring and `Highcharts` for dashboard analytics.
- **Application Layer (FastAPI)** : Serves static files, manages user sessions, processes file uploads, and coordinates speech transcription and LLM grading engines.
- **Worker Layer (Celery & Redis)** : Offloads heavy media tasks (such as Cloudinary uploads and Whisper transcription processing) to background workers.
- **Data Storage Layer (PostgreSQL/SQLite)** : Manages database tables and relations. SQLite is used for local storage, while PostgreSQL is used in production.

4.2 Component Interaction & Communication Pipelines

Communication between system components is designed to ensure reliable data transfer :

- **HTTP REST Endpoints** : Used for candidate registration, authentication, resume uploads, profile updates, and admin actions.
 - **WebRTC Connection Protocol** : Used to establish WebRTC media channels, allowing candidates to stream and record webcam feeds directly to the backend container.
 - **Celery Event Broker** : Coordinates async background tasks. Redis tracks task states and communicates completions back to the FastAPI application layer.
 - **External API Connections** : Interconnects the backend with external providers like Groq Whisper, Cloudinary, and Gemini APIs.
-

4.3 Database Relational Schema Layout

The database uses a relational schema defined via SQLAlchemy :

- ****Users Table**** : Stores candidate accounts and profiles. Includes fields for credentials, experience levels, skills, contact details, and proctoring violation notes.
- ****Interview Sessions Table**** : Stores individual session metrics. Linked to the Users table via a foreign key on the username. Tracks the prompt questions, candidate responses, emotional states, scores, and video links.

SQLAlchemy ORM relationships (relationship) enable cascading deletions and optimize database queries to prevent performance bottlenecks.

4.4 SQLite-PostgreSQL Dual DB Sync Architecture

To maintain high availability and prevent data loss, the system implements a dual database synchronization architecture :

- **Local SQLite Staging** : Acts as a local storage fallback when external databases are unreachable.
- **Production PostgreSQL** : Serves as the primary production database.

When running in PostgreSQL mode, transaction events (like registration, profile updates, and grading) are mirrored to SQLite. If the PostgreSQL database goes offline, the application continues to write to SQLite. Once connection is restored, the ``migrate_to_postgres.py`` utility can be used to synchronize the databases.

4.5 WebRTC Signaling & Real-Time Peer-to-Peer Protocol

The system uses WebRTC for efficient real-time media recording :

- **Signaling Exchange** : The client initiates a WebRTC connection request (SDP Offer) via the `/api/webrtc/offer`` endpoint.
- **Host Negotiation** : The backend processes the request using `aiortc`` and returns a local SDP Answer, establishing a direct peer-to-peer channel.
- **Media Streaming** : Once the connection is established, the client streams video and audio tracks directly to the backend container, where a `MediaRecorder`` instance saves the feed to a WebM file.

Chapter 5: Backend Implementation Details

5.1 Entrypoints: Deep Dive into main.py Core Orchestrator

The core orchestrator of the system is `main.py`, which manages application startup and endpoint routing  :

- ****Bootstrap & Middleware****  : Configures FastAPI, sets up CORS middleware to allow cross-origin requests, and mounts static asset directories.
- ****Database Initialization****  : Runs migrations to dynamically add columns to existing tables, and seeds default admin credentials (`admin` / `admin123`) on startup.
- ****Request Handling****  : Implements JSON error handlers to catch unexpected exceptions, return structured error payloads, and log debug stack traces.

5.2 Database Configuration & SQLAlchemy Engine Mapping

The database configuration layer (`database.py`) maps connections based on active environment variables :

- **URL Sanitization** : Auto-translates legacy `postgres://` connection strings to `postgresql://` to prevent SQLAlchemy driver errors.
- **Connection Pooling** : Sets `pool\_pre\_ping=True` in production to automatically test database connections and reconnect if a timeout occurs.
- **SQLite Concurrency** : Configures SQLite with `check\_same\_thread=False` to support concurrent read and write operations during local development.

5.3 Resume Extraction: Heuristic Parsing Algorithms 🔍

The parser (`modules/resume_parser.py`) uses `pdfplumber` to extract and process resume text 🔍:

- **Keyword Matching** 📄: Scans document text against a database of technical skills using regular expression word boundaries (`\b``) to prevent false positive matches.
- **Degree Mapping** 🎓: Analyzes educational terms using priority matching to classify qualifications into specific disciplines (e.g., mapping ``B.Tech in Computer Science`` to ``B.Tech CSE``).
- **Heuristic Extraction** 🔍: Extracts email addresses, phone numbers, and location info using regular expression patterns, and estimates age from parsed date-of-birth formats.

5.4 Semantic Search: FAISS Vector Space Vetting

The question retriever (`modules/question_retriever.py`) uses semantic vector search to select interview questions :

- **Model Loading**  : Initializes a lightweight SentenceTransformer model (`all-MiniLM-L6-v2`) to generate vector embeddings.
- **Embedding Computation**  : Converts the question bank into 384-dimensional dense vectors and stores them in a FAISS Flat L2 index.
- **Querying & Retrieval**  : Converts candidate skills into query embeddings to retrieve the most semantically relevant technical questions from the database.

5.5 Question Generation Algorithms & Duplication Vetting 🎯

If machine learning libraries cannot be loaded, the system falls back to a rule-based selection algorithm 🎯:

- ****Dynamic Count Selection**** 📊: Adjusts the number of questions based on experience levels: freshers receive 4-5 questions, while experienced candidates receive 6-7.
- ****Skill Distribution**** 📁: Selects technical questions across parsed skills in a round-robin format, pulling questions from the local question bank.
- ****Duplication Vetting**** 🔍: Queries past database records on startup to ensure candidates are not presented with questions they have previously answered.

5.6 Voice Activity Detection (VAD) & Noise Modeling

The voice activity detection module (`modules/vad_detector.py`) identifies active speech in candidate recordings :

- **Silero VAD Engine** : Uses a pre-trained Silero VAD model via PyTorch to evaluate audio streams and calculate speech probability.
 - **Energy-Based Fallback** : If PyTorch or the VAD model fails to initialize, the system falls back to a Root Mean Square (RMS) energy-based algorithm.
 - **Amplitude Check** : Measures volume thresholds in real-time, classifying inputs with an RMS value greater than `0.025` as active speech.
-

5.7 Speech Transcription: Groq Whisper Client Integration

Audio transcription is handled by the transcriber module (`modules/whisper_transcriber.py`) :

- **API Request Construction** : Sends recorded WebM/WAV files to the Groq Whisper API endpoint via POST requests, using JWT authentication.
- **Model Parameters** : Configures the request to use the `whisper-large-v3-turbo` model with temperature set to `0.0` to ensure highly accurate, deterministic transcriptions.
- **Error Handlers** : Catches and logs connection errors, fallback issues, and invalid file formats, returning fallback responses if transcription fails.

5.8 LLM Evaluation Engine: Gemini Client Configuration 🤖

The grading module (`modules/answer\_analyzer.py`) uses LLMs to evaluate candidate answers 🤖:

- ****Gemini Integration**** 🌐: Connects to the Gemini 1.5 Flash API using standard Python network libraries (`urllib`).
- ****Prompt Engineering**** 🗣️: Instructs the model to evaluate the candidate's answer for accuracy, technical depth, and delivery tone, taking their emotional state into account.
- ****Structured JSON Output**** 📄: Configures the API payload to enforce a strict JSON schema (`score` and `feedback`) to ensure predictable responses.

5.9 LLM Evaluation Engine: GPT-4o & Ollama Clients

If the primary Gemini API is unavailable, the grading engine falls back to alternative LLMs :

- **GPT-4o Client** ⚡ : Initializes the OpenAI SDK to request response evaluations using GPT-4o, enforcing structured JSON formatting.
- **Ollama Local Integration** ⚙️ : Connects to local Llama 3 models via local endpoints if the `USE_OLLAMA` environment flag is enabled.

These fallback paths ensure the system remains operational and can grade responses even during external API outages.

5.10 Local TF-IDF Offline Cosine Similarity Grader Fallback

If all cloud LLMs and local APIs are unreachable, the system falls back to offline text processing :

- **TF-IDF Vectorization** : Uses Scikit-Learn to convert the question and the candidate's transcribed answer into TF-IDF vector representations.
- **Cosine Similarity** : Calculates the cosine similarity score between the two vectors, using this value to calculate a base grading score.
- **Dynamic Adjustments** : Awards scoring bonuses for answer length and adjusts the final grade based on client-side emotion metrics.

5.11 Background Task Workers & Redis Broker Setup

To handle resource-heavy operations without slowing down the web interface, the system offloads processing to background tasks :

- **Celery Configuration**  : Configures a Celery worker application (`celery\_app`) using Redis as the message broker.
 - **Task Offloading**  : Routes long-running operations (like transcribing audio and uploading media to Cloudinary) to background worker processes.
 - **Thread Isolation**  : Isolates audio processing from main API routes, keeping web endpoints fast and responsive during heavy concurrent interviews.
-

5.12 Media Storage: Cloudinary API Client Integration

Media files are managed by the storage client module (`storage.py`) :

- **SDK Configuration**  : Initializes the Cloudinary SDK using API key and secret environment variables.
 - **Chunked Uploads**  : Uses `upload_large` to upload video responses, tracking progress and returning secure HTTPS URLs.
 - **Local Fallback**  : If Cloudinary credentials are not configured, the system saves files directly to the local directory (`/static/videos`).
-

Chapter 6: Front-End UI Engine & Real-Time Analytics

6.1 Style Systems: Core Visual Tokens & CSS Layouts

The system's user interface is styled using vanilla CSS to ensure fast load times and clean, responsive designs :

- **Theme Configurations**  : Implements a modern dark theme using slate backgrounds, cyan highlights, and glassmorphism styling.
- **Layout Systems**  : Uses flexbox and grid layouts to adapt the interface to different screen sizes and device types.
- **Interactive States**  : Enhances interactive elements with CSS hover transitions, focus glowing borders, and progress indicator animations.

6.2 Live Visual Calibration & Eye Gaze Proctoring

Proctoring checks are processed locally in the candidate's browser during the interview session  :

- **Landmark Detection** : Uses `Face-API.js` to identify facial landmarks and calculate eye positions relative to the camera.
 - **Focus Analysis** : Measures gaze drift. If the candidate looks away from the screen for more than two seconds, the system logs a violation.
 - **Visibility Observers** : Uses document visibility listeners to detect when the candidate switches tabs or minimizes the browser window.
-

6.3 Webcam Feed Emotion Recognition Matrix 🤖

The simulation page maps candidate emotional states in real-time using client-side libraries 🤖:

- **Neural Network Parsing** 🤖: Evaluates visual expressions via `Face-API.js` to calculate raw scores for emotions like happy, sad, angry, and surprised.
- **UI Visualizers** 📊: Renders live emotion levels as horizontal progress bars that update in real-time alongside the video stream.
- **Dominant Sentiment Logging** 🤖: Records the dominant emotion during each response and sends the data to the backend database.

6.4 Media Session Recording: WebRTC Chunk Capturing ■

Answering responses are captured using browser-level media APIs ■:

- **Webcam Access** ■: Requests user media streams (audio and video) at startup and displays the webcam feed in the UI.
 - **Chunked Recording** 📁: Uses the MediaRecorder API to record response snippets, caching data blocks as WebM blobs.
 - **File Compilation** 📁: Compiles the data chunks on response submission, sending the complete WebM file to the backend via POST requests.
-

6.5 User Experience (UX) Flow & Candidate Interfaces

The interface is divided into distinct stages to guide candidates through the evaluation :

- ****Calibration Stage****  : Guides candidates through account creation, calibration details, and resume uploading.
 - ****Simulation Stage**** : Displays active questions, webcam feeds, proctoring metrics, and time limits.
 - ****Reporting Stage**** : Renders average scores, emotional trends, integrity ratings, and detailed technical feedback cards.
-

Chapter 7: Recruiter Management & Dashboards

7.1 Recruiter Interfaces: Deep Dive into manager.html

The recruiter interface (`manager.html`) provides tools to manage candidates and review interview results :

- **Candidate Grid** : Displays applicant details, including usernames, statuses, experience levels, domains, and integrity flags.
- **Interactive Filters** : Provides filter options to search candidate lists by domain, experience level, or status.
- **Action Controls** : Allows admins to approve or reject applications, grant/revoke simulation access, and delete profiles.

7.2 Analytics Engine: Highcharts Integration

The admin dashboard displays visual metrics using the Highcharts library :

- **Candidate Rankings** : Renders 3D column charts showing average scores across candidates to help identify top performers.
- **Status Distributions** : Displays pie charts representing candidate approval rates (Pending, Approved, Rejected).
- **Domain Vetting** : Shows bar charts detailing the distribution of technical domains and skills across the applicant pool.

7.3 Candidate Dossiers, Transcripts & Evaluation Cards

Clicking a candidate's row opens a detailed dossier view :

- **Dossier Metrics** : Displays contact details (extracted from the resume), qualifications, detected skills, and proctoring histories.
- **Transcripts** : Lists questions, transcribed answers, emotional states, scores, and feedback generated by the system.
- **Media Replay** : Embeds HTML5 video players to allow recruiters to review recorded video responses for each question.

7.4 Outreach Systems: Gmail & WhatsApp Integration 📢

The admin panel integrates outreach tools to simplify communication with candidates 📢:

- **Gmail Integration** 📧: Generates pre-formatted email templates containing interview feedback, scores, and application updates.
- **WhatsApp Integration** 💬: Creates pre-filled WhatsApp message links to allow recruiters to contact candidates directly.

These links open in the recruiter's browser, simplifying candidate outreach and status updates.

7.5 System Access Flags & Administrative Controls

Admins can manage candidate access to the interview room in real-time :

- **Simulation Access** : Toggling the access flag to `revoke` immediately blocks the candidate from entering the interview room.
- **Proctoring Flags** : Toggling candidate status to `Compromised` flags candidates who switched tabs or looked away from the camera.
- **Data Cleanup** : Allows admins to delete specific candidate records or clear the entire database.

Chapter 8: DevOps, Containerization & Operations ■■

8.1 Dockerfile Configurations & Build Strategies ■

The application is containerized using Docker to ensure consistent environment setups ■:

- **Base Image** ■: Uses `python:3.10-slim` as the base image to keep the container lightweight.
- **System Dependencies** ⚙️ : Installs `ffmpeg` inside the container for Whisper audio processing, along with C compilers for libraries.
- **Startup Command** ⚡ : Starts the FastAPI web server using Uvicorn, mapping ports and binding hosts for incoming requests.

8.2 Docker Compose Orchestration & Service Isolation ■■

For production environments, the services are managed using Docker Compose ■■:

- **db Service** : Runs a PostgreSQL 15 database to store candidate profiles, schemas, and interview data.
- **redis Service** : Launches a Redis instance to handle messaging and routing between FastAPI and Celery.
- **worker Service** : Builds the application environment and runs the Celery worker daemon to process background tasks.

8.3 Celery Configurations & Worker Tuning

The background worker is optimized to handle resource-heavy media processing  :

- **Task Routing**  : Routes Whisper transcription tasks to the Celery worker queue to prevent slowing down the main API.
 - **Upload Tasks**  : Offloads video uploads to Cloudinary to a background thread.
 - **Concurrency Limits**  : Sets concurrency limits based on host CPU resources to prevent OOM errors.
-

8.4 SQLite to PostgreSQL Migration Scripts

The migration utility script (`migrate_to_postgres.py`) handles database migrations :

- **Data Extraction**  : Connects to the local SQLite database and extracts candidate profiles and interview records.
 - **Bulk Insertion**  : Validates records and bulk-inserts them into the production PostgreSQL database.
 - **Sequence Sync**  : Adjusts primary key sequences in PostgreSQL to prevent ID collisions on subsequent inserts.
-

Chapter 9: Testing, Verification & Quality Assurance

9.1 Unit Testing Framework & Implementation ■

We validated system functions using modular Python testing frameworks ■:

- **Resume Parsing Tests** 📄: Verifies name, email, and skill extraction on sample resume formats.
- **LLM Fallback Tests** 🗣️: Simulates API failures to confirm the system falls back to offline TF-IDF similarity evaluations.
- **VAD Performance** 🎧: Verifies voice activity detection calculations under different audio noise profiles.

9.2 System Integration Test Cases

Integration testing verified that the various system components interact correctly :

- **Authentication Flows** : Verifies the registration, login, and token-based authentication workflows.
- **End-to-End Simulation** : Tests the full interview workflow, from profile calibration to results generation.
- **Proctoring Logs** : Verifies that tab switches and camera obstructions are logged and affect the final score.

9.3 AI Grading Consistency & Transcription Latency 🗨️

We evaluated grading performance and processing times 🗨️:

- ****Grading Reliability**** 🗨️: Verifies that LLM evaluations return structured scores and feedback within expected parameters.
- ****Transcription Speeds**** 🗨️: Measures Whisper processing times for response audio, ensuring results return within two seconds.
- ****Fallback Correctness**** ⚙️: Verifies that offline TF-IDF evaluations remain active during internet connectivity outages.

9.4 Visual Proctoring Validation & Obfuscation

The proctoring system was tested to verify cheating detection  :

- **Face-API Validation** : Confirms that Face-API.js correctly flags missing faces when the webcam feed is obstructed.
 - **Gaze Deviation** : Verifies that looking away from the camera for more than three seconds triggers proctoring logs.
 - **Tab Swapping** : Confirms that browser tab switches trigger proctoring alerts and flag candidate records.
-